

DSA Progress & Revision Analyzer

Python Essentials — Evaluated Course Project

A modular command-line application for DSA problem tracking, progress analytics, and spaced-repetition revision scheduling.

Project type: Academic Python application

Storage: SQLite

Runtime: Python standard library

Testing: pytest

Verified test result: 42 passed

1. Introduction

Preparing for DSA interviews requires more than recording whether a problem was solved. A useful tracker should show progress patterns, identify weaker topics, and remind the learner when previously solved problems should be revised. This project combines persistent problem tracking, analytics, and a deterministic spaced-repetition scheduler in one CLI application.

2. Problem Statement

Learners often maintain DSA progress in scattered notes or simple lists. Such systems do not provide topic-level analysis, solving streaks, revision history, or scheduled revision reminders. The DSA Progress & Revision Analyzer addresses this by storing structured problem data and deriving actionable progress and revision information.

3. Objectives

- Maintain a structured DSA problem database.
- Validate problem and solve-date data.
- Provide progress and topic analytics.
- Identify weak topics using solved-count thresholds.
- Schedule revisions using 1, 3, 7, 14 and 30-day intervals.
- Record revision history and generate reports.
- Provide automated tests and maintainable modular architecture.

4. Functional Requirements

ID	Requirement
FR1	Add, view, update and delete DSA problems
FR2	Search and filter by title, topic, difficulty, status and platform
FR3	Persist data using SQLite
FR4	Calculate status, difficulty, topic, weekly and streak analytics
FR5	Identify weak topics
FR6	Schedule and record spaced-repetition revisions
FR7	View due, upcoming and revision-history information
FR8	Generate TXT, JSON and CSV reports
FR9	Import sample problems from JSON
FR10	Return clear validation errors and non-zero failure codes

5. Non-Functional Requirements

Usability — readable CLI output and simple commands.

Reliability — validation, exception handling and automated tests.

Maintainability — separate modules for storage, business logic, analytics, scheduling and reporting.

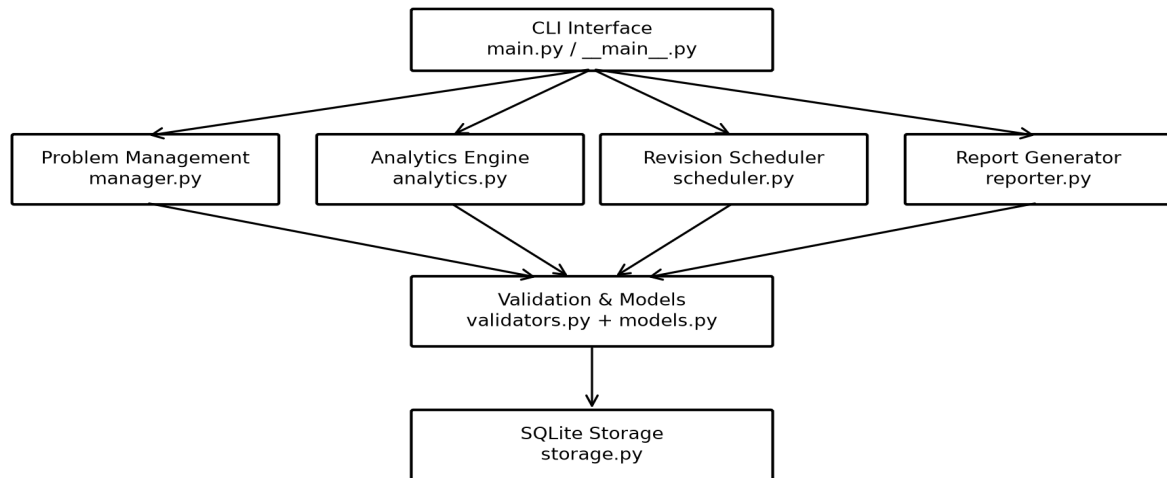
Performance — lightweight SQLite queries and indexes.

Portability — Python 3.10+ and SQLite.

Testability — business logic can be tested independently using temporary databases.

6. System Architecture

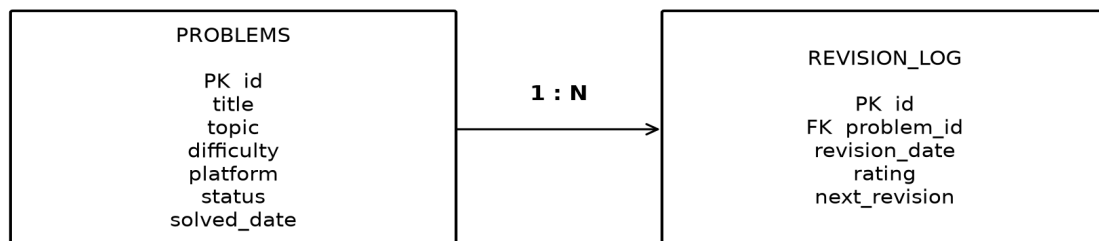
DSA Progress & Revision Analyzer - Architecture



The CLI layer delegates business operations to ProblemManager and RevisionScheduler. Validators protect data quality, Database provides persistence, Analytics derives progress metrics, and Reporter produces export files.

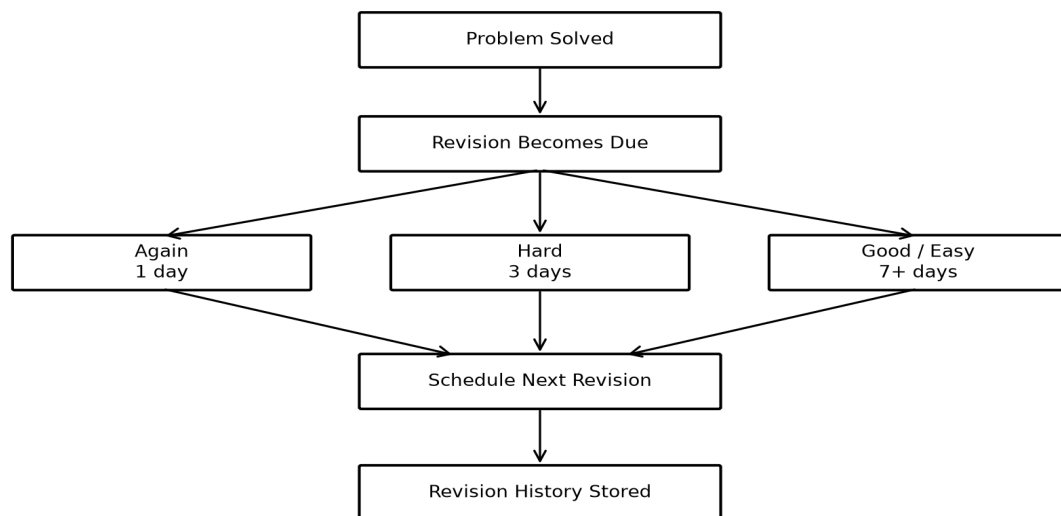
7. Database / ER Design

DSA Tracker - ER Diagram



8. Revision Scheduling Workflow

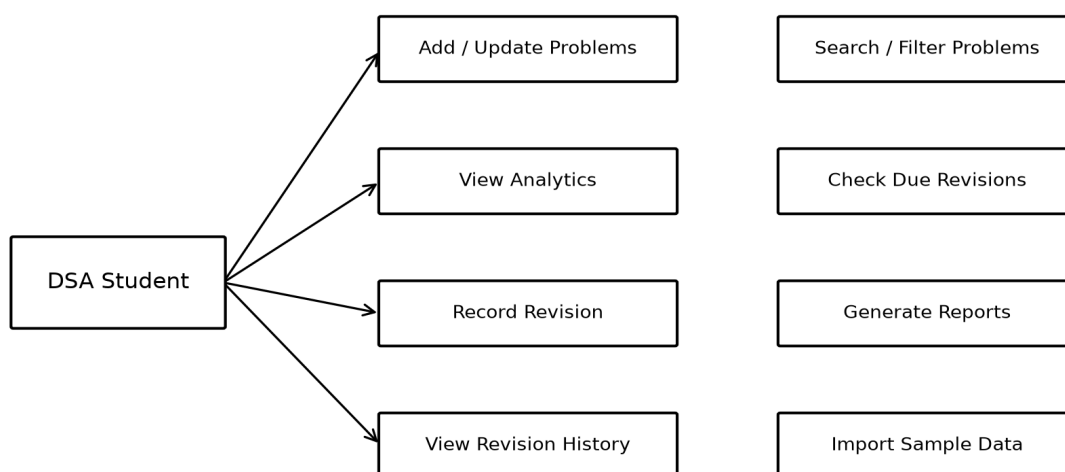
Spaced Repetition Revision Workflow



A solved problem receives an initial revision date one day after its solve date. A recorded revision then advances or resets the revision level according to the rating. The scheduler uses deterministic intervals of 1, 3, 7, 14 and 30 days.

9. Use Case Design

DSA Tracker - Use Case Diagram



10. Implementation Modules

Module	Responsibility
models.py	Problem data model
validators.py	Input, date and status validation
storage.py	SQLite schema, CRUD and indexes
manager.py	Problem business logic and import handling
analytics.py	Statistics, streaks, weekly progress and weak topics
scheduler.py	Spaced-repetition scheduling, due/upcoming queries
reporter.py	TXT, JSON and CSV report generation
logger_config.py	File-based logging configuration
main.py	CLI parser and command handlers
tests/	Automated functional and module tests

11. Design Decisions

SQLite was selected to provide persistent storage without a separate server. The layered structure separates concerns and makes the scheduling and analytics engines independently testable. The runtime deliberately uses the Python standard library to keep setup lightweight; pytest is used only for automated testing.

12. Testing and Verification

Final automated test result: 42 passed, 0 failed. Tests cover problem creation, duplicate detection, validation, updates, deletion, imports, revision history, analytics, streaks, weak topics, scheduler behavior, due/upcoming queries and invalid inputs.

Progress Analytics

```
Progress Analytics

DSA PROGRESS SUMMARY
=====
Total Problems: 50

Status:
  Solved: 30
  Attempted: 11
  Unsolved: 9

Difficulty:
  Easy: 15
  Medium: 27
  Hard: 8

Topics:
  Arrays: 8
  Strings: 7
  Dynamic Programming: 6
  Graphs: 6
  Linked List: 5
  Searching: 5
  Trees: 5
  Stack & Queue: 3
  Two Pointers: 3
  Hashing: 2

Weekly Progress:
  Week starting 2026-08-10: 0 solved
  Week starting 2026-08-17: 6 solved
  Week starting 2026-08-24: 5 solved
  Week starting 2026-08-31: 3 solved
```

Due Revisions

```
Due Revisions

DUE REVISIONS
=====
[10] Maximum Product Subarray | Dynamic Programming | Due: 2026-08-19
[15] Largest Rectangle in Histogram | Stack & Queue | Due: 2026-08-19
[29] Edit Distance | Dynamic Programming | Due: 2026-08-19
[33] Invert Binary Tree | Trees | Due: 2026-08-19
[11] 3Sum | Arrays | Due: 2026-08-22
[39] Group Anagrams | Strings | Due: 2026-08-22
[47] Maximum Subarray | Arrays | Due: 2026-08-26
[24] Reverse Linked List | Linked List | Due: 2026-08-28
[31] Rotting Oranges | Graphs | Due: 2026-08-29
[27] Clone Graph | Graphs | Due: 2026-08-30
[40] Merge Two Sorted Lists | Linked List | Due: 2026-08-30
[25] Move Zeroes | Two Pointers | Due: 2026-09-04
[21] Implement Queue Using Stacks | Stack & Queue | Due: 2026-09-06
[30] Koko Eating Bananas | Searching | Due: 2026-09-07
[8] Valid Parentheses | Strings | Due: 2026-09-08
[41] Coin Change | Dynamic Programming | Due: 2026-09-08
[5] Find Minimum in Rotated Sorted Array | Searching | Due: 2026-09-13
[13] 3Sum Closest | Two Pointers | Due: 2026-09-14
[37] Serialize and Deserialize Binary Tree | Trees | Due: 2026-09-15
[12] Binary Tree Level Order Traversal | Trees | Due: 2026-09-16
[16] Longest Consecutive Sequence | Hashing | Due: 2026-09-17
[9] Container With Most Water | Arrays | Due: 2026-09-18
[18] Word Ladder | Graphs | Due: 2026-09-18
[36] Validate Binary Search Tree | Trees | Due: 2026-09-21
[23] Remove Nth Node From End of List | Linked List | Due: 2026-09-23
[45] Two Sum II | Two Pointers | Due: 2026-09-24
[4] Daily Temperatures | Stack & Queue | Due: 2026-09-27
[2] Trapping Rain Water | Arrays | Due: 2026-09-30

Total due: 28
```

Upcoming Revisions

Upcoming Revisions

UPCOMING REVISIONS – NEXT 7 DAYS

```
=====
[2] Trapping Rain Water | Due: 2026-09-30
[1] Number of Islands | Due: 2026-10-01
[32] Two Sum | Due: 2026-10-01
```

Total upcoming: 3

Revision Result

Revision Result

Revision recorded successfully.
Problem: Number of Islands
Rating: good
Revision Level: 1
Next Revision: 2026-10-03

Revision History

Revision History

REVISION HISTORY – PROBLEM 1

```
=====
2026-09-30 | good
```

13. Error Handling Improvements

The final version rejects future solve dates, prevents contradictory status/date combinations, keeps application logs in the log file instead of duplicating them in console output, and returns a non-zero process exit code when a command fails. A clean CLI error therefore behaves correctly for both the user and the shell.

14. Demo Data

The sample dataset contains 50 DSA problems. It intentionally includes solved, attempted and unsolved records, at least one weak topic, and solved dates that demonstrate both due and upcoming revisions. This makes the scheduler visible immediately after importing the sample data.

15. Challenges and Learnings

Key implementation challenges included coordinating SQLite persistence with revision history, keeping analytics consistent with validated status/date data, and making the revision engine deterministic enough for automated testing. The project demonstrates modular Python design, CLI development, database handling, validation, testing and basic algorithmic scheduling.

16. Future Enhancements

Possible extensions include a graphical dashboard, configurable revision intervals, richer performance trends, cloud synchronization and multi-user authentication.

17. References

Python documentation; SQLite documentation; pytest documentation; spaced-repetition concepts used as the basis for the fixed revision-interval scheduler.

Conclusion

The final DSA Progress & Revision Analyzer provides a complete CLI workflow from problem entry and validation through analytics, scheduled revision, history and reporting, with automated tests and supporting design documentation.